
РАБОЧИЙ ПРОТОКОЛ И ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
"ЛР 4: [C++ и UNIX]: C++ PROCESSES / THREADS"

Группа: Z3344
Студенты: Поздеев Дмитрий
Преподаватель: Маслов И.

К работе допущен:
Работа выполнена:
Отчет принят:

1 Цель работы

- Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

2 Задачи, решаемые при выполнении работы

- C++ SEQUENCE Последовательные вычисления
- C++ THREADS Параллельные вычисления через потоки
- C++ PROCESS Параллельные вычисления через процессы
- LOG Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

3 Код

3.1 C++ SEQUENCE

Листинг 1: Код на плюсах

```

#include <iostream>
#include <chrono>

using namespace std;
using namespace std::chrono;

double formula1(double x) {
    return x * x - x * x + 4 * x - 5 * x + x + x;
}

double formula2(double x) {
    return x + x;
}

double formula3(double res1, double res2) {
    return res1 + res2 - res1;
}

void run_sequential(int N, double x) {
    auto start = high_resolution_clock::now();

    for (int i = 0; i < N; ++i) {
        double res1 = formula1(x);
        double res2 = formula2(x);
        double res3 = formula3(res1, res2);
        asm volatile("" : "+g" (res3)); //
    }

    auto end = high_resolution_clock::now();
    duration<double> diff = end - start;
    cout << "Sequential_time_for_" << N << "_iterations:_ " << diff.count() << "
}

int main() {
    double x = 5.0;
    run_sequential(10000, x);
    run_sequential(100000, x);
    return 0;
}

```

3.2 C++ THREADS

Листинг 2: код на плюсах

```

#include <iostream>
#include <chrono>
#include <thread>
#include <future>

```

```

using namespace std;
using namespace std::chrono;

double formula1(double x) {
    return x * x - x * x + 4 * x - 5 * x + x + x;
}

double formula2(double x) {
    return x + x;
}

void run_threads(int N, double x) {
    double res1, res2;
    double time1, time2, time3;

    auto future1 = async(launch::async, [&]() {
        auto start = high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            res1 = formula1(x);
        }
        auto end = high_resolution_clock::now();
        return duration_cast<duration<double>>(end - start).count();
    });

    auto future2 = async(launch::async, [&]() {
        auto start = high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            res2 = formula2(x);
        }
        auto end = high_resolution_clock::now();
        return duration_cast<duration<double>>(end - start).count();
    });

    time1 = future1.get();
    time2 = future2.get();

    auto start3 = high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        double res3 = res1 + res2 - res1;
        asm volatile("" : "+g" (res3));
    }
    auto end3 = high_resolution_clock::now();
    time3 = duration_cast<duration<double>>(end3 - start3).count();

    cout << "Threads_time_for_" << N << "_iterations:\n";
    cout << "Formula1:_ " << time1 << "_sec\n";
    cout << "Formula2:_ " << time2 << "_sec\n";
    cout << "Formula3:_ " << time3 << "_sec\n";
    cout << "Total:_ " << max(time1, time2) + time3 << "_sec\n";
}

```

```
int main() {
    double x = 5.0;
    run_threads(10000, x);
    run_threads(100000, x);
    return 0;
}
```

3.3 C++ PROCESS

Листинг 3: код на плюсах

```
#include <iostream>
#include <chrono>
#include <unistd.h>
#include <sys/wait.h>

using namespace std;
using namespace std::chrono;

struct Result {
    double value;
    double duration;
};

double formula1(double x) {
    return x * x - x * x + 4 * x - 5 * x + x + x;
}

double formula2(double x) {
    return x + x;
}

void run_processes(int N, double x) {
    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid_t pid1 = fork();
    if (pid1 == 0) {
        close(pipe1[0]);
        Result res;
        auto start = high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            res.value = formula1(x);
        }
        auto end = high_resolution_clock::now();
        res.duration = duration_cast<duration<double>>(end - start).count();
    }
}
```

```

        write(pipe1[1], &res, sizeof(res));
        close(pipe1[1]);
        exit(0);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        close(pipe2[0]);
        Result res;
        auto start = high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            res.value = formula2(x);
        }
        auto end = high_resolution_clock::now();
        res.duration = duration_cast<duration<double>>(end - start).count();
        write(pipe2[1], &res, sizeof(res));
        close(pipe2[1]);
        exit(0);
    }

    close(pipe1[1]);
    close(pipe2[1]);

    Result res1, res2;
    read(pipe1[0], &res1, sizeof(res1));
    read(pipe2[0], &res2, sizeof(res2));

    close(pipe1[0]);
    close(pipe2[0]);

    waitpid(pid1, nullptr, 0);
    waitpid(pid2, nullptr, 0);

    auto start3 = high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        double res3 = res1.value + res2.value - res1.value;
        asm volatile("" : "+g" (res3));
    }
    auto end3 = high_resolution_clock::now();
    double time3 = duration_cast<duration<double>>(end3 - start3).count();

    cout << "Processes_time_for_" << N << "_iterations:\n";
    cout << "Formula1:_ " << res1.duration << "_sec\n";
    cout << "Formula2:_ " << res2.duration << "_sec\n";
    cout << "Formula3:_ " << time3 << "_sec\n";
    cout << "Total:_ " << max(res1.duration, res2.duration) + time3 << "_sec\n";
}

int main() {
    double x = 5.0;

```

```

    run_processes(10000, x);
    run_processes(100000, x);
    return 0;
}

```

3.4 Работа с Git

Листинг 4: Клонирование репозитория и создание структуры

```

git clone https://github.com/PozdeevDA/my_labs.git
cd my_labs
mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
git checkout -b dev
git push -u origin dev
git branch -d main
git push origin --delete main

```

Листинг 5: Скрипт для переноса из dev в stg:

```

git add .
git commit -m "last_fix"
git push origin dev

```

4 Выводы

При выполнении вычислений параллельные потоки демонстрируют наибольшую эффективность благодаря разделяемой памяти и минимальным накладным расходам, сокращая общее время за счёт одновременного выполнения формул 1 и 2, тогда как процессы уступают из-за затрат на создание и синхронизацию через IPC, а последовательный подход, хотя и предсказуем, наименее оптимален из-за линейной зависимости времени от числа итераций, при этом этап формулы 3 остаётся узким местом во всех реализациях, так как требует последовательной обработки результатов.